Week 3 LLM Monitoring Experiment

-D. Mohit Venkat Sai

-2520030106

Aim

To build a provider based LLM client that sends prompts to AI models, measures each interaction, and stores the results in MongoDB.

Objectives

- Connect a Python program to hosted and local LLM providers.
- Generate responses using Gemini and Ollama.
- Record prompts, responses, token counts, and cost information.
- Verify that interactions from both providers are saved in MongoDB.

Software and Tools

- Python and Visual Studio Code
- Ollama with the llama3.2 model
- Gemini API with the gemini-3.8-flash model
- MongoDB Community Server and MongoDB Compass
- Python packages: google-genai, python-dotenv, pymongo, and requests

System Design

The program uses a common provider interface so the client can work with different LLM providers.

- **OllamaProvider** sends prompts to the local Ollama service.
- **GeminiProvider** sends prompts to the hosted Gemini API.
- **LLMClient** calls the selected provider and prepares interaction metrics.
- **MongoLogger** saves each interaction in MongoDB.

Procedure

1. Installed Ollama and downloaded the llama3.2 model.
2. Confirmed that the MongoDB service was running.
3. Created the Week 3 Python project and installed the required packages in the virtual environment.
4. Added the provider, client, cost calculation, and MongoDB logging code to the project.

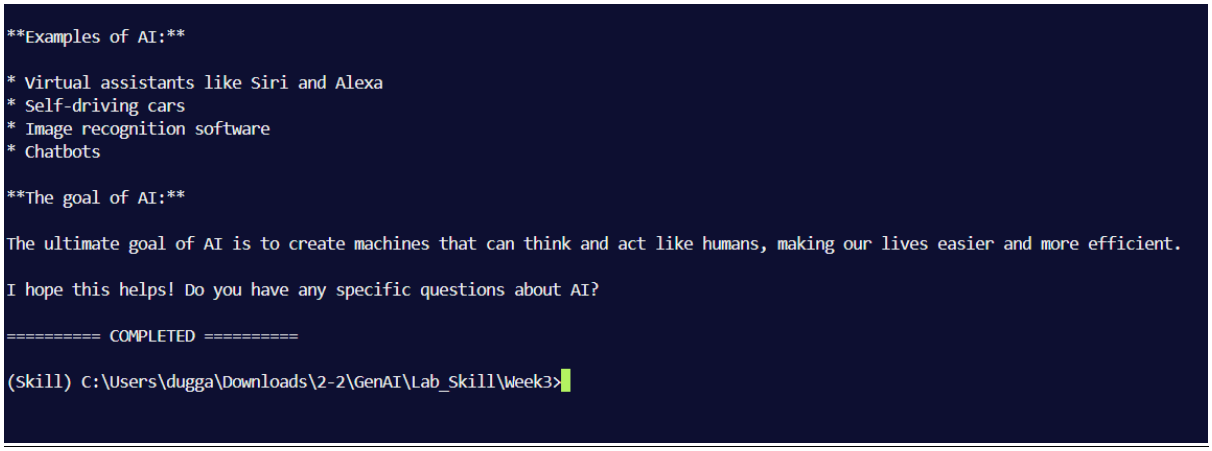
5. Ran the program and selected Ollama. Entered a prompt and received a response from llama3.2.
6. Ran the program again and selected Gemini. Entered a prompt and received a response from gemini-3.8-flash.
7. Opened MongoDB Compass and verified that both interactions were saved.

Results

Both provider runs completed successfully. MongoDB Compass showed two documents in the llm_calls collection: one for Ollama and one for Gemini.

Ollama Output





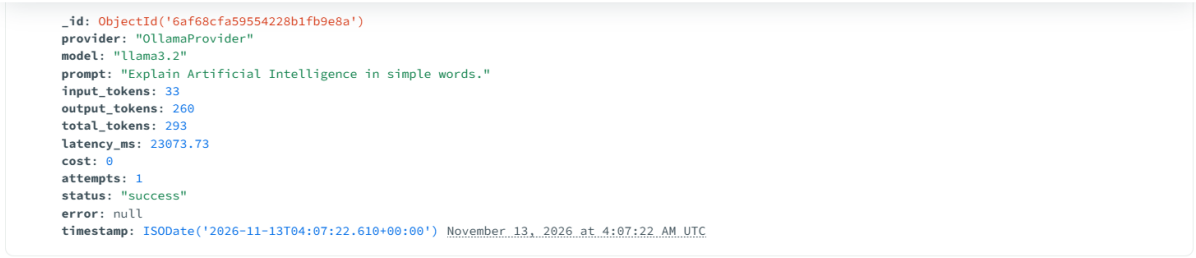
Gemini Output



Provider	Model	Prompt	Result
Ollama	llama3.2	“Explain AI”	Response generated and logged
Gemini	gemini-3.8-flash	“Explain AI”	Response generated and logged

The Ollama run reported 28 prompt tokens, 711 completion tokens, 739 total tokens, and a cost of $0.00. The Gemini document also contains token and cost fields; their values were not expanded in the Compass screenshot.

The records were saved in the MongoDB database **genai_logs**, collection **llm_calls**. Each record includes a timestamp, provider, model, prompt, response, token information, and cost information.





Conclusion

The experiment successfully demonstrated provider based LLM monitoring. The same program generated responses using both a local Ollama model and the hosted Gemini API, then saved both interactions in MongoDB. The Compass records confirm that the logging workflow works for both providers.